Sistema FCT



Guía de incorporación y operaciones de desarrollo para el Sistema FCT, una aplicación web híbrida que gestiona prácticas de Formación en Centros de Trabajo (FCT).

> Docker Compose es el entorno de desarrollo recomendado. Esta configuración no es un despliegue de producción: los puertos y secretos de ejemplo son locales y deben cambiarse antes de compartir el entorno.

1. Tecnologías

- **Backend:** PHP 8.2, Apache y arquitectura MVC personalizada.
- **Frontend:** Vue 3, TypeScript, Pinia y Vite como islas progresivas sobre vistas HTML renderizadas por PHP.
- **Persistencia:** MariaDB 10.4 mediante PDO y consultas preparadas.
- **Seguridad de archivos:** ClamAV (TCP INSTREAM); los documentos se guardan en almacenamiento privado, fuera de `public/`.
- **Infraestructura:** Docker Compose v2 con volúmenes persistentes para la base de datos, almacenamiento y firmas antivirus.
- **Documentación:** portal VitePress con diagramas Mermaid.

2. Requisitos previos

- Git.
- Docker Desktop actualizado con Docker Compose v2 (`docker compose` , no el ejecutable antiguo `docker-compose`).
- Node.js 22–24 y npm para ejecutar Vite, pruebas de tipos o VitePress directamente en el host.
- PowerShell 5.1 o posterior para los comandos de esta guía en Windows.

El contenedor `app` incluye PHP 8.2, Apache y las extensiones PDO MySQL y mbstring; no es necesario instalar PHP en Windows para el flujo Docker.

La arquitectura se organiza en tres capas: presentación (HTML/PHP y pequeñas islas Vue), aplicación (rutas, middleware, controladores y reglas PHP) y datos (MariaDB y almacenamiento privado). Vue no sustituye la navegación MVC ni la autorización del servidor.

3. Preparar el entorno

Clona el repositorio:

```
git clone https://github.com/campusedx/Practicas_Desarrollo.git
```

Entra en la carpeta recién creada:

```
Set-Location .\Practicas_Desarrollo
```

Crea `.env` con los valores locales de referencia; Compose lo carga automáticamente desde la raíz:

```
Copy-Item .env.example .env
```

Abre `.env` para cambiar puertos o contraseñas si entran en conflicto con otros servicios:

```
notepad .env
```

Variables incluidas en la plantilla:

Variable	Predeterminado	Uso
`APP_PORT`	`8080`	Puerto HTTP de PHP/Apache en el host.
`APP_ENV`	`development`	Entorno de ejecución; el bypass de ClamAV sólo se permite con este valor.
`FCT_CLAMAV_ENABLED`	`false` en las plantillas locales	`false` omite explícitamente el escaneo sólo en desarrollo; usa `true` para probar ClamAV.
`PMA_PORT`	`8082`	Puerto local de phpMyAdmin; se publica sólo en `127.0.0.1`.
`FCT_DB_NAME`	`sistema_fct`	Base de datos de desarrollo.
`FCT_DB_USER`	`fct`	Usuario que utiliza la aplicación y el formulario de acceso de phpMyAdmin.
`FCT_DB_PASS`	`fct_local_dev`	Contraseña del usuario de aplicación MariaDB.
`MARIADB_ROOT_PASSWORD`	`root_local_dev`	Contraseña administrativa de MariaDB, reservada para tareas locales de administración.
`FCT_VITE_DEV_SERVER`	vacío	URL pública del servidor Vite que PHP debe incluir durante desarrollo, por ejemplo `http://localhost:5173`.
`VITE_PORT`	`5173`	Puerto de Vite cuando se ejecuta desde el perfil Docker `frontend-dev`.

Los valores de `.env.example` son exclusivamente para desarrollo local; sustitúyelos por claves propias y no compartas ni confirmes `.env`.

Compose crea automáticamente el volumen privado `fct_storage` y lo monta como `/var/lib/fct/storage`, que es el valor de `FCT_STORAGE_PATH` dentro del contenedor. El Dockerfile prepara el directorio y sus permisos; no se debe mover a `public/`.

Si ejecutas PHP directamente con XAMPP en vez de Docker, configura `FCT_STORAGE_PATH` como una ruta absoluta a un directorio existente, con permisos de escritura para Apache y separado del document root. La aplicación rechaza rutas inexistentes o dentro de `public/`.

4. Iniciar y detener la infraestructura

Construye la imagen de la aplicación e inicia Apache/PHP, MariaDB, ClamAV y phpMyAdmin en segundo plano:

```
docker compose up -d --build
```

Comprueba que los servicios estén activos y revisa su estado de salud:

```
docker compose ps
```

ClamAV puede tardar varios minutos en descargar sus primeras firmas. El contenedor `app` espera a que MariaDB esté saludable, pero puede iniciar mientras ClamAV prepara sus firmas; si el análisis está habilitado, una carga se rechazará hasta que el antivirus responda correctamente. Consulta su salida si todavía está inicializando:

```
docker compose logs --tail=50 clamav
```

Comprueba que el almacenamiento privado exista y sea escribible por PHP:

```
docker compose exec app sh -c 'test -d "$FCT_STORAGE_PATH" && test -w "$FCT_STORAGE_PATH"'
```

Abre la aplicación en http://localhost:8080. Para detener los contenedores sin borrar datos:

```
docker compose down
```

Los volúmenes de base de datos, archivos subidos, firmas ClamAV y dependencias Docker persisten al detener los contenedores.

5. Base de datos, cuentas de desarrollo y phpMyAdmin

En la primera inicialización de un volumen MariaDB vacío, Compose importa `database/sistema_fct.sql` y aplica, en orden, las migraciones de `database/migrations/001_...sql` a `010_...sql`. La migración 008 añade el indicador de disponibilidad horaria indiferente; la 009 incorpora DAM a las especialidades existentes; la 010 añade detalles persistidos a las ofertas. La inicialización oficial de MariaDB sólo procesa esos archivos cuando crea una base de datos nueva; no vuelve a ejecutarlos en cada arranque.

Las ofertas históricas aprobadas no tienen horas exactas inferibles a partir de sus antiguos turnos amplios; se excluyen del matching y del catálogo hasta republicarlas con horario semanal. No asignes horarios aproximados automáticamente.

Para una base ya inicializada, no vuelvas a importar el esquema ni ejecutes migraciones existentes indiscriminadamente: varias no son idempotentes. Examina `docker compose ps` y los registros de MariaDB antes de diagnosticar una instalación incompleta. La migración 009 es idempotente y puede aplicarse a una base existente para añadir DAM. Para un entorno nuevo creado fuera de Compose, ejecuta el esquema y las migraciones una sola vez, en orden, contra esa base vacía:

```
Get-Content -Raw .\database\sistema_fct.sql | docker compose exec -T db sh -c 'mariadb --user="$MARIADB_USER" --
        password="$MARIADB_PASSWORD" "$MARIADB_DATABASE"'
```

Este comando carga el esquema inicial en la base seleccionada por las variables de Compose; úsalo sólo si el esquema aún no existe:

```
Get-ChildItem .\database\migrations\*.sql | Sort-Object Name | ForEach-Object {
    Get-Content -Raw $_.FullName | docker compose exec -T db sh -c 'mariadb --user="$MARIADB_USER" --
        password="$MARIADB_PASSWORD" "$MARIADB_DATABASE"'
}
```

El bucle aplica las migraciones SQL en orden alfabético; no lo ejecutes sobre un volumen que ya haya recibido esas migraciones durante la inicialización de Compose.

Para una base existente que ya tiene las migraciones anteriores a la 007, aplica las migraciones 007 y 008 en orden, y sólo si no se aplicaron previamente:

```
Get-Content -Raw .\database\migrations\007_horarios_ofertas.sql | docker compose exec -T db sh -c 'mariadb --
        user="$MARIADB_USER" --****** "$MARIADB_DATABASE"'
```

Este comando crea la tabla de horarios exactos de las ofertas sin volver a ejecutar migraciones antiguas; reinicia el servicio `app` después de aplicarlo para que su comprobación de salud detecte la tabla.

Para una base que ya recibió la migración 007, aplica únicamente el campo de disponibilidad indiferente:

```
Get-Content -Raw .\database\migrations\008_disponibilidad_indiferente.sql | docker compose exec -T db sh -c 'mariadb
        --user="$MARIADB_USER" --****** "$MARIADB_DATABASE"'
```

La migración 008 añade `es_indiferente` de forma idempotente y migra el valor anterior de `turno` sólo la primera vez; la 009 inserta DAM si aún no existe. En una base que ya recibió la 008, aplica la migración 009 para habilitar DAM:

```
Get-Content -Raw .\database\migrations\009_desarrollo_aplicaciones_multiplataforma.sql | docker compose exec -T db sh
        -c 'mariadb --user="$MARIADB_USER" --****** "$MARIADB_DATABASE"'
```

El matching omite la cobertura horaria diaria para el alumnado indiferente, pero sigue exigiendo fechas, ciclo, oferta aprobada con franjas y capacidad.

La migración 010 añade título, descripción, turno, modalidad, ubicación, requisitos y tecnologías a las ofertas existentes. En una base que ya recibió la migración 009, aplícala una vez:

```
Get-Content -Raw .\database\migrations\010_detalles_ofertas.sql | docker compose exec -T db sh -c 'mariadb --
        user="$MARIADB_USER" --****** "$MARIADB_DATABASE"'
```

La migración es idempotente. Los datos de ofertas históricas reciben valores predeterminados compatibles; una descripción/ubicación no se inventa para esos registros.

Genera las cinco cuentas de prueba actuales (Administrador, Coordinador, Observador, Empresa y Alumno) sólo en la base local autorizada:

```
docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php scripts/crear_cuentas_demo.php
```

El script imprime contraseñas aleatorias una sola vez. **Cada ejecución renueva todas las contraseñas demo y deja inválidas las anteriores**; guarda la salida de forma segura. También crea para el alumno demo disponibilidad de lunes a viernes, de 09:00 a 17:00, con franjas exactas. No ejecutes este comando en producción.
`database/seeds/001_sample_data.sql` es un conjunto legado de datos de muestra y no se ejecuta automáticamente ni sustituye al generador seguro de cuentas.

Consulta la matriz de roles y permisos para conocer el acceso restringido al catálogo y el alcance de solo lectura del perfil Observador.

Abre phpMyAdmin en http://localhost:8082. Inicia sesión con el servidor `db`, puerto `3306`, base `sistema_fct` y las credenciales `FCT_DB_USER` / `FCT_DB_PASS` de `.env` (por defecto, `fct`). El usuario `root` y `MARIADB_ROOT_PASSWORD` deben reservarse para administración local cuando sea necesaria; no uses phpMyAdmin para exponer el servicio a la red.

El puerto MariaDB publicado para clientes del host es `127.0.0.1:3307`; dentro de Compose, la aplicación y phpMyAdmin se conectan al nombre de servicio `db:3306`.

6. Recorrido funcional: disponibilidad, matching y seguimiento

El sistema compara ofertas y expedientes del mismo ciclo formativo, por ejemplo DAW con DAW y DAM con DAM. Para que una asignación sea válida, la oferta debe estar aprobada, tener franjas semanales publicadas y plazas disponibles; el periodo de disponibilidad del alumno debe cubrir el periodo completo de la oferta. El servidor vuelve a validar todos esos criterios al asignar, aunque la pantalla ya haya mostrado candidatos.

Los ciclos soportados incluyen **DAW** (Desarrollo de Aplicaciones Web) y **DAM** (Desarrollo de Aplicaciones Multiplataforma). La especialidad se relaciona por ID en la base de datos, y la migración 009 incorpora DAM sin duplicarlo. Una oferta guarda un único ciclo para preservar el matching actual.

La publicación recoge título, descripción, plazas, fechas, turno, franjas semanales de cinco minutos, modalidad, ubicación, requisitos y tecnologías. La modalidad presencial e híbrida requiere ubicación. La revisión y los listados muestran los detalles persistidos; el matching sigue usando la misma relación por ciclo, fechas y horarios.

En el portal del alumno, la disponibilidad tiene tres pasos de configuración:

1. Indica las fechas de inicio y fin del periodo.
2. Elige una de estas opciones horarias:
    - **Franjas exactas:** marca cada día y define su hora de inicio y fin. Puedes aplicar un horario general a varios días y luego editar cada día.
    - **Disponibilidad indiferente:** indica que aceptas cualquier horario de oferta; las franjas por día quedan desactivadas. Fechas y ciclo siguen siendo obligatorios.
3. Envía el formulario para guardar. La isla Vue muestra de forma reactiva el estado de guardado o el error; no guarda cambios automáticamente. Sin JavaScript, el formulario HTML/PHP mantiene el envío y la validación del servidor.

Con franjas exactas, cada intervalo ofertado `[inicio, fin]` debe quedar contenido en un intervalo de disponibilidad del mismo día (`disponibilidad.inicio <= oferta.inicio` y `disponibilidad.fin >= oferta.fin`). La opción indiferente omite únicamente esa comparación horaria: no omite el ciclo, las fechas, la aprobación, las franjas publicadas ni el control de plazas. Las ofertas históricas sin horarios semanales exactos no participan en el matching hasta volver a publicarse con su horario.

Administrador y Coordinador registran calificaciones de prácticas cerradas; se admiten notas de **0.00 a 10.00**, con hasta dos decimales y sin caracteres alfanuméricos. El alumno consulta sus propias notas. El tema claro/oscuro se administra mediante Pinia, conserva la elección en `localStorage` y ofrece un selector visual SVG. El perfil Observador es de solo lectura y recibe datos anonimizados, sin acceso a documentos privados. La política completa está en Roles y permisos.

Rol	Permisos principales
Administrador	Gestión de usuarios, ofertas/convenios y decisiones de revisión; asignaciones y calificaciones.
Coordinador	Seguimiento, asignaciones, cierre de prácticas, calificaciones y consultas; no aprueba ofertas o convenios.
Observador	Sólo lectura, con datos anonimizados y sin documentos privados; `POST /logout` sólo invalida su sesión.
Alumno	Su perfil, disponibilidad, asignaciones y calificaciones, más información de su empresa asignada.
Empresa	Sus convenios, sedes y ofertas; no decide sus revisiones.
Invitado	Inicio, registro, autenticación y recuperación; las áreas protegidas requieren sesión.

El botón de tema claro/oscuro sigue la preferencia del sistema si todavía no hay una elección guardada; al cambiarlo manualmente, la elección queda en el almacenamiento local del navegador.

Antivirus en desarrollo local

Compose ejecuta ClamAV y PHP se conecta a él por la red interna del proyecto. Las plantillas locales establecen `FCT_CLAMAV_ENABLED=false` para permitir el desarrollo si ClamAV no está disponible; este modo registra una advertencia y omite el escaneo sólo cuando `APP_ENV=development`. Para probar el escaneo real, establece `FCT_CLAMAV_ENABLED=true` y espera a que `docker compose ps` muestre `clamav` saludable. En producción, deja el escaneo habilitado: configuración inválida, fallo de conexión, firmas caducadas, timeout o detección de amenaza rechazan la carga.

Si necesitas comprobar el comportamiento de fallo cerrado, establece `FCT_CLAMAV_ENABLED=true`, detén ClamAV sólo en un entorno local desechable (`docker compose stop clamav`), confirma que una carga se rechaza y vuelve a iniciarlo (`docker compose up -d clamav`). No desactives el análisis fuera de desarrollo ni expongas el puerto interno del scanner.

7. Acceso a la aplicación y herramientas

Servicio	Dirección local
Aplicación PHP	http://localhost:8080 (o `APP_PORT`)
phpMyAdmin	http://localhost:8082 (o `PMA_PORT`)
Vista previa de disponibilidad (sólo desarrollo)	http://localhost:8080/index.php?action=preview-disponibilidad
Vista previa de publicar oferta (sólo desarrollo)	`http://localhost:8080/dev/preview-publicar-oferta`
Vite HMR	http://localhost:5173 cuando se inicia el frontend
Portal VitePress	http://localhost:5174 con el comando de desarrollo de documentación

El acceso a páginas y acciones dentro de la aplicación continúa gobernado por `routes/web.php` y el rol de la sesión; conocer la URL no concede permisos.

8. Frontend y Vite HMR

Instala las dependencias fijadas en `package-lock.json` para trabajar con Vite y Vue desde el host:

```
npm ci
```

Define la URL del servidor Vite en `.env` para que las vistas PHP incluyan el cliente HMR y los módulos frontend durante desarrollo:

```
FCT_VITE_DEV_SERVER=http://localhost:5173
```

Aplica el cambio de entorno recreando el servicio PHP si ya estaba iniciado:

```
docker compose up -d app
```

Inicia Vite en el puerto usado en la URL anterior:

```
npm run dev
```

Vite sirve HMR en http://localhost:5173. Si el puerto está ocupado, libera el puerto o cambia `VITE_PORT` (perfil Docker) y `FCT_VITE_DEV_SERVER` para que la URL configurada coincida con la URL accesible desde el navegador.

También puedes iniciar el servidor Vite en un contenedor con el perfil opcional:

```
docker compose --profile frontend-dev up -d
```

El perfil instala dependencias con `npm ci`, ejecuta Vite escuchando en todas las interfaces del contenedor y publica `VITE_PORT` (5173 por defecto). Para regresar al bundle de producción incluido en la imagen, detén el perfil y vacía `FCT_VITE_DEV_SERVER` en `.env`.

Las vistas siguen renderizándose en PHP y conservan sus formularios como fallback; Vue se monta sólo en elementos marcados `data-vue-island`. Las acciones JSON usan el contrato `{ success, data }` / `{ success, error }`, envían `X-CSRF-Token` y no reemplazan la navegación MVC por rutas SPA.

Cuando los cambios de una isla Vue no aparecen

La isla de disponibilidad actual se llama `AvailabilitySchedule` (archivo `frontend/src/islands/AvailabilitySchedule.vue`); el registro de `frontend/src/main.ts` descubre automáticamente los componentes Vue y la vista PHP debe usar el mismo valor en `data-vue-island`.

- **Desarrollo con HMR:** define `FCT_VITE_DEV_SERVER=http://localhost:5173` en `.env`, recrea PHP con `docker compose up -d app` para cargar esa variable, y ejecuta `npm run dev` en el host (o inicia el perfil `frontend-dev`). Comprueba que el navegador puede abrir `http://localhost:5173`.
- **Bundle de producción Docker:** `npm run build` genera `public/build` en el host, pero no actualiza una imagen PHP ya creada. Reconstruye y recrea la aplicación con `docker compose up -d --build app`; la imagen copia el bundle durante el build.
- **Comprueba el montaje:** abre la consola y la pestaña Network del navegador, fuerza recarga sin caché y confirma que se solicita el entrypoint de Vite o el bundle `build/assets/main-*.js`. Revisa que el HTML entregue el marcador `data-vue-island` y que su nombre corresponda a un archivo en `frontend/src/islands/`.
- **Si HMR conserva una versión antigua:** compara el módulo servido en `http://localhost:5173/frontend/src/islands/AvailabilitySchedule.vue` con el archivo del host. Si no coinciden, reinicia `docker compose restart frontend-dev` (o vuelve a iniciar el proceso `npm run dev`) para limpiar las

transformaciones en memoria de Vite; después recarga el navegador sin caché. `docker compose down -v` no es necesario para invalidar HMR.

- **Aísla la vista:** Compose establece `APP_ENV=development` por defecto. Usa `http://localhost:8080/index.php?action=preview-disponibilidad` para mostrar la isla con datos de prueba sin iniciar sesión; configura `APP_ENV=production` en despliegues para que la ruta no se registre.

Valida el frontend tras modificar componentes con `npm run typecheck` y `npm run build`. No hace falta purgar volúmenes para desplegar cambios de interfaz: basta con HMR o reconstruir la imagen de la aplicación.

9. Portal de documentación VitePress

Usa `npm ci` desde la raíz si aún no instalaste las dependencias Node. Para evitar competir con Vite HMR en el puerto 5173, ejecuta la documentación en el puerto 5174:

```
npm run docs:dev -- --port 5174
```

Abre http://localhost:5174; si el puerto indicado también está ocupado, VitePress asignará otro puerto disponible y lo mostrará en la terminal. El portal está en `docs/` y renderiza los diagramas Mermaid.

Compila el portal para verificar páginas Markdown, configuración y diagramas:

```
npm run docs:build
```

Sirve localmente la última compilación de VitePress:

```
npm run docs:preview -- --port 4173
```

10. Verificaciones y pruebas

Comprueba los tipos de los componentes e islas Vue/TypeScript:

```
npm run typecheck
```

Genera el bundle de producción y ejecuta de nuevo la comprobación de tipos:

```
npm run build
```

Ejecuta las pruebas de seguridad de almacenamiento y análisis de archivos en el contenedor PHP (las pruebas usan un directorio temporal y no requieren la base de datos):

```
$tests = (Resolve-Path .\tests).Path -replace '\\', '/'
docker compose run --rm --no-deps -v "${tests}:/var/www/html/tests:ro" app php tests/file_security.php
```

`tests/run.php` es una prueba de integración y debe apuntar exclusivamente a una base de datos desechable cuyo nombre empiece por `fct_test_`; configura `FCT_TEST_DB_NAME` y `FCT_DB_NAME` con el mismo nombre antes de ejecutarla. No la ejecutes contra `sistema_fct` ni una base que quieras conservar.

Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones

> ⚠️ **ADVERTENCIA: restablecimiento destructivo.** `docker compose down -v` elimina todos los volúmenes nombrados de este proyecto Compose, no sólo el de MariaDB. Se perderán la base de datos (`db_data`), los archivos privados subidos (`fct_storage`), las firmas de ClamAV (`clamav_db`) y las dependencias Docker (`node_modules` y `frontend_node_modules`). Es irreversible; usa este procedimiento sólo en un entorno local desechable y después de guardar cualquier dato que quieras conservar.

Alternativa más segura si sólo necesitas reinicializar la base de datos: detén el proyecto, elimina únicamente el volumen `db_data` y vuelve a iniciarlo. Así se conservan el almacenamiento privado, ClamAV y las dependencias:

```
docker compose down
$dbVolume = (docker compose config --format json | ConvertFrom-Json).volumes.db_data.name
docker volume rm $dbVolume
docker compose up -d --build
```

Para restablecer **todos** los datos locales después de añadir cambios de esquema o migraciones, sigue estos tres pasos:

1. Elimina los contenedores y todos los volúmenes del proyecto:

   ```
   docker compose down -v
   ```

2. Reconstruye e inicia los servicios. MariaDB importará el esquema y ejecutará las migraciones al crear su volumen vacío:

   ```
   docker compose up -d --build
   ```

3. Si necesitas las cuentas demo, créalas de nuevo en la base local:

   ```
   docker compose exec -e FCT_ENABLE_DEMO_SEED=1 -e FCT_DEMO_SEED_ALLOW_DOCKER=1 app php
           scripts/crear_cuentas_demo.php
   ```

   El script renueva las contraseñas demo en cada ejecución; guarda la salida de forma segura. No lo ejecutes en producción.

11. Solución de problemas frecuentes

Puertos ocupados

Los puertos predeterminados son `8080` (aplicación), `8082` (phpMyAdmin), `3307` (MariaDB del host) y `5173` (Vite HMR). Cambia `APP_PORT` , `PMA_PORT` o `VITE_PORT` en `.env` para los puertos publicados configurables; el puerto de MariaDB se cambia editando el mapeo explícito `127.0.0.1:3307:3306` en `docker-compose.yml` . Si al abrir `http://localhost:8081` aparece Jenkins u otro servicio, eso indica que ese puerto ya está ocupado: establece `PMA_PORT=8082` (o cualquier puerto host libre) en `.env` y ejecuta `docker compose up -d` para publicar phpMyAdmin en el puerto elegido. Si VitePress comparte 5173 con Vite, usa el puerto 5174 recomendado.

Lista los puertos y estados publicados por Compose:

```
docker compose ps
```

Almacenamiento no escribible

Comprueba desde PHP que la ruta privada está montada y permite escritura:

```
docker compose exec app sh -c 'printf "FCT_STORAGE_PATH=%s\n" "$FCT_STORAGE_PATH"; test -d "$FCT_STORAGE_PATH" &&
        test -w "$FCT_STORAGE_PATH"'
```

En Docker, `FCT_STORAGE_PATH` debe seguir siendo `/var/lib/fct/storage`; si usas XAMPP, crea el directorio local fuera de `public/` y asigna permisos a la cuenta que ejecuta Apache.

MariaDB o ClamAV aún no saludables

Consulta el estado de los servicios:

```
docker compose ps
```

Consulta los últimos mensajes de MariaDB y ClamAV para encontrar errores de inicio o el progreso de descarga de firmas:

```
docker compose logs --tail=100 db clamav
```

El primer arranque de ClamAV puede tardar; espera a que indique `healthy` antes de investigar como fallo.

Para permitir cargas de prueba locales sin escaneo, configura explícitamente `APP_ENV=development` y `FCT_CLAMAV_ENABLED=false`; los registros de PHP indicarán que se omitió ClamAV. Si ambos valores son correctos y se requiere análisis real, establece `FCT_CLAMAV_ENABLED=true` y revisa la conectividad interna a `clamav:3310`. Nunca uses el bypass en producción.

Ciclos DAW y DAM

Si no aparece DAM en una base ya existente, aplica una vez `database/migrations/009_desarrollo_aplicaciones_multiplataforma.sql` a esa base. En un volumen MariaDB nuevo, Compose ejecuta esa migración automáticamente. El matching compara IDs de especialidad: DAW sólo coincide con DAW y DAM con DAM.

Disponibilidad o controles de contraseña

Las horas se envían como `HH:MM` desde `<input type="time">` y el servidor acepta también `HH:MM:SS`; si una carga manual falla, revisa que días, indicador de disponibilidad indiferente y token CSRF correspondan al contrato. En formularios de contraseña, el botón de ojo cambia la visibilidad sin enviar el formulario; si no cambia, comprueba que `public/assets/js/password-toggle.js` esté cargado y que el campo siga siendo `type="password"` al cargar la página.

Reiniciar sin borrar datos

Reinicia los contenedores sin borrar volúmenes:

```
docker compose down
docker compose up -d --build
```

Para volver a inicializar la base de datos tras cambios de esquema o migraciones, consulta Restablecimiento de Base de Datos por Cambios de Esquema o Migraciones y revisa la advertencia sobre el alcance de `docker compose down -v`.

Más información

- Hoja de ruta de migración
- Flujo de datos y ciclo de vida
- Diagrama completo de arquitectura
- Configuración y fuentes de verdad: `routes/web.php`, `config/`, `database/` y `docker-compose.yml`.